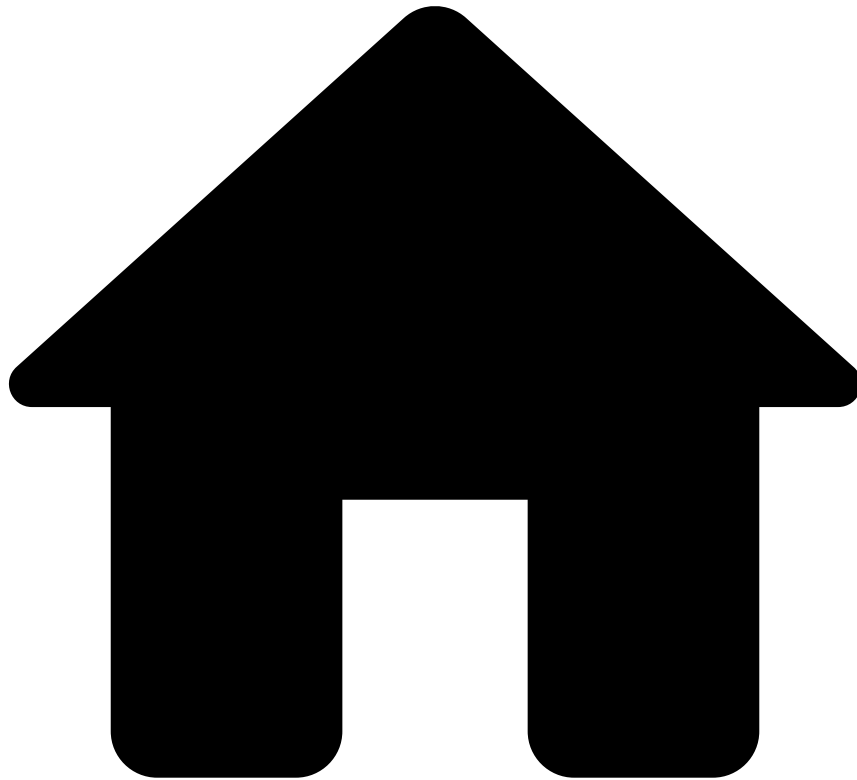


•



- [Core Concepts](#)
- [Models](#)
- [Model Training](#)
- Microsoft LightGBM Provider

On this page

Microsoft LightGBM Provider

Was this page helpful? 🗨️

Light Gradient Boosting Machine or LightGBM is a gradient boosting framework for several types of tree-based learning algorithms.

In this topic you can learn about the parameters and the usage of the Microsoft LightGBM Provider in Pulse.

Microsoft LightGBM

The biggest advantage of Microsoft LightGBM is that training can be very fast with low memory usage and good performance. Whilst Microsoft LightGBM models support ranking, classification and regression tasks in a distributed manner, Pulse's LightGBM machine learning provider only supports binary classification tasks.

One of its limitations is that LightGBM only receives numerical data (even for categorical features). This means that strings must undergo a process called *String Indexing*, where a mapping of 1:1 from a string to a numerical value is performed. Of course, this mapping must be maintained consistent at all times (training and scoring).

You can choose from several boosting types:

- **Gradient Boosting Decision Trees (GBDT)**
- **Gradient Based One Side Sampling (GOSS)**: faster than the one above but may yield slightly lower predictive performance.
- **Dropouts meet Multiple Additive Regression Trees (DART)**: can be much slower but might yield higher predictive performance as dropout can yield less over-fitting.
- **Random Forest (RF)**: not recommended as it is not a true Random Forest but uses boosting as well - use our FZRF instead.

LightGBM in Pulse

Pulse can train LightGBM models and even import external ones using this provider. Only numerical and categorical features are supported, string fields are not used in the LightGBM models.

Numerical features are passed along to the model without transformation, whilst categorical features go through an automatic string-indexing algorithm.

Categorical features handling in Pulse for LightGBM models

Since LightGBM can only take in numerical values, categorical features are fully supported by running a simple **string-indexing algorithm at both training and scoring time**, by transforming the category string value into the *n*th index in the lexicographically-sorted list of categories of a categorical variable.

See the example for the string indexing applied to a categorical feature value:

Property	Value
Input value	"adam"
Categorical feature categories	["John" , "42" , "Adam" , "zoe" , "adam" , "Jane"]
Categories in lexicographic order	["42" , "Adam" , "Jane" , "John" , "adam" , "zoe"]
Output value	4

Notice that indexes start from 0 and for ASCII character codes, numbers come before letters, and uppercase letters come before lowercase ones.

This arbitrary choice does not affect analytical performance as LightGBM does not consider any order for categorical features.

Training a model in Pulse

To train a model in Pulse, start a new model training just like any other model type. If you need more details, see the [Training a Model](#) page for a refresher.

All categorical features in the training (and scoring) schema are automatically mapped using the algorithm explained in the section above before being sent to LightGBM. The same mapping is automatically used during scoring.

Numerical features are not transformed.

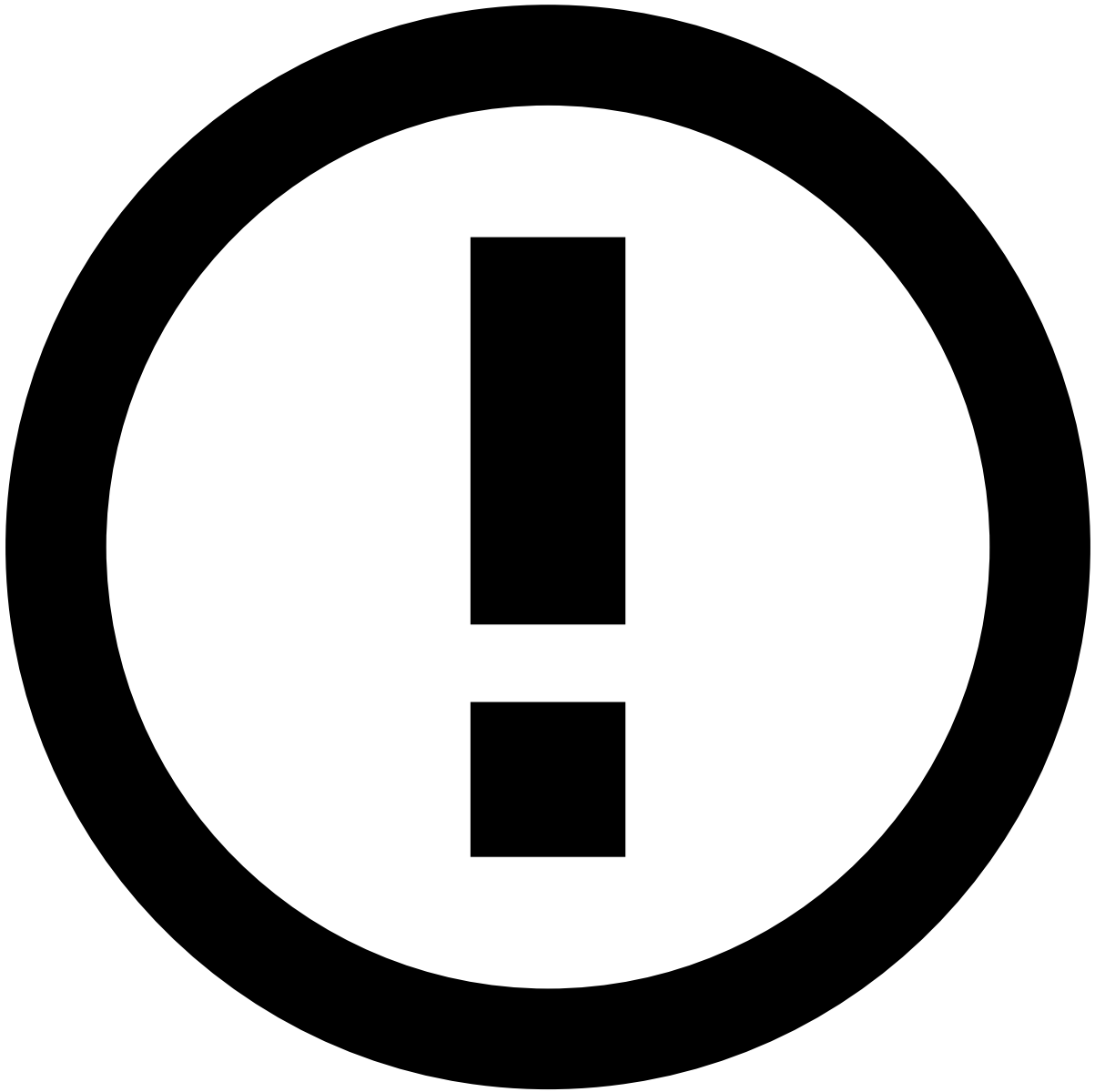
Importing a model to Pulse

You can import a LightGBM model trained outside Pulse as well.

Model import path

To import a model there are two options available:

1. Import the model from a model file path:
 1. The path needs to point to a valid LightGBM model file that LightGBM exports in .txt format.
 2. Since you are not providing the schema, you must define the schema inside the Pulse UI.
2. Import the LightGBM model from a folder path:
 1. The folder pointed to must contain at least the following files:
 1. `LightGBM_model.txt` (mandatory) - the model file as in 1.a.
 2. `model.json` (optional) - a JSON designating the model schema
 2. Since `model.json` is optional:
 1. If not present in the folder, you have to define the schema inside the Pulse UI, as in 1.b.
 2. If present, it will be used for the model schema, unless you override it as in 1.b.



info

Regarding section 2.a.ii, see this [example JSON file](#) for binary classification with both numerical and categorical features.

Importing models with categoricals

If an imported model had categoricals in training, you had to index them, and thus that same indexing must now be performed during scoring, in whichever environment is used.

If indexing during training and scoring does not match, the model scores will be affected.

You have several options and must ensure consistency of those transformations:

1. If you declare, in the Pulse model schema, a variable to be categorical, lexicographic order will automatically be used by Pulse for string indexing during scoring:
 - The only way to have correct scores is to train externally in the first place with the same indexing algorithm, otherwise the model's categorical features in training and scoring differ from each other.

2. If you declare, in the Pulse schema, a variable to be numerical (and at the input it is a string that you had to string-index), you're going to have to do the same exact string-indexing transformation for scoring yourself:
 - Make sure you use the same indexing during training and scoring, but that in scoring, such transformations are reliable across environments, with tolerable latencies and memory usage for your use case.

Parameters

Check the configuration parameters in the [official documentation of LightGBM](#). In Pulse, all parameters follow those specifications, including default values. Handling missing values is always enabled.

Note that there are some parameters that only apply to GOSS and DART boosting modes. Such parameters always have respectively GOSS or DART in their names.

Logging configuration

If you want to know in more detail the progress of the process, you can check the [LightGBM Logging Configuration](#) page to configure logging to suit your needs.

Learn more

- [Microsoft LightGBM official documentation](#)
- [Microsoft LightGBM code on Github](#)
- [FAQ on Gradient Boosting Frameworks](#)